dimos_huazhijian
多-Agent 开发流水线配置执行说明书

供 Cursor / Codex Agent 阅读与执行

版本：v1.0

日期：2026-04-24

用途：把当前仓库逐步配置为“提需求 -> 写代码 -> 本地测试 -> Push -> PR -> Review -> Merge”的稳定流水线

适用对象：Cursor 本地 Agent、Codex Review Agent、仓库维护者

一句话目标： 先把本地 Cursor + GitHub CI + 自动 PR / Auto-merge 跑通；Codex 先承担 Review / 第二视角检查，等第一条闭环稳定后，再升级到更多云端自动化。

1. 文档定位与执行目标

这不是普通介绍文档，而是给 Agent 的执行说明书。读完后，Agent 应直接进入仓库检查、补齐配置、跑验证、提交 PR，并把不能自动完成的部分整理成明确的人类待办。

最终目标

- 用户只需要提出需求，不再手工逐项完成 Git、测试、PR、Review 和 Merge。
- 仓库内形成稳定的“本地开发 + 云端检查”的流水线：需求 -> 代码变更 -> 本地验证 -> Push 分支 -> 创建 PR -> 云端 CI / Review -> 自动合并到 main。
- 第一次实施优先追求一次成功，不追求最花哨的全云端自治。

已知条件

- 本地仓库名称：dimos_huazhijian。
- 当前主要开发环境：Cursor。
- 用户具备：Cursor Plus、ChatGPT Pro（可使用 Codex）。
- 代码托管目标：GitHub。
- 技术栈未显式给出；Agent 必须通过仓库文件自行识别，不要先问用户。

成功标准（Definition of Done）

1. 仓库中存在唯一、可重复执行的验证入口，例如 scripts/verify.sh 或 make verify。
2. 仓库中存在给 Agent 使用的说明文件，例如 AGENTS.md、Cursor 项目规则、PR 模板等。
3. GitHub Actions 跑的检查与本地验证尽量一致，避免“本地绿、云端红”。
4. 所有代码变更都通过功能分支和 PR 进入 main，不允许直接推送 main。
5. 当 required checks 通过后，可以自动合并到 main；如果暂时无法全自动，Agent 必须明确指出最后一跳的人工动作。

2. Agent 分工与职责边界

多-Agent 不是让多个 Agent 同时改同一分支，而是把职责拆开：一个负责推进任务，一个负责改代码，一个负责修失败，一个负责评审。必须避免多个 Agent 同时对同一分支做无约束写入。

角色	核心任务	可以做	不可以做	产出
Orchestrator Agent（建议由 Cursor 本地 Agent 承担）	检查仓库、制定计划、分配步骤、汇总结果	识别技术栈；决定先做什么；协调编码/调试/验证	未经验证直接合并；同时发起多个互相冲突的改动	执行计划、阶段总结

Coding Agent	在最小变更范围内实现功能或补齐配置	修改代码、补脚本、补文档、补测试	越权做大规模重构；跳过验证	代码改动、测试更新
Debug / Test Agent	处理 lint / build / test 失败	读取报错；定位原因；做最小修复	引入需求外功能；伪造通过结果	失败原因与修复说明
Git / PR Agent	标准化 Git 和 PR 流程	建分支、提交、push、创建 PR、开启 auto-merge	直接 push main；强推覆盖别人分支	PR 链接、提交信息
Review Agent（建议由 Codex 承担）	对 PR 做第二视角审查	找回归风险、缺测试、潜在破坏性改动	替代 CI；替代唯一真相验证命令	Review 意见、阻塞项

重要边界

- 同一时间只有一个 Agent 对当前工作分支拥有写权限；Review Agent 只提意见，不直接改写同一分支，除非 Orchestrator 明确接管。
- 任何角色都不能跳过验证脚本；验证脚本是整条流水线的唯一真相来源。
- 如果 GitHub 网页设置无法通过命令或 API 自动完成，Agent 不允许假装已配置完成，必须生成“人类一次性待办清单”。

3. 执行原则与总流程

总原则

- 先本地、后云端：第一次闭环先让本地 Cursor 能稳定做完“改代码 + 本地测 + Push + PR”。
- 先标准化、后自动化：先把 verify 入口和仓库规则做稳定，再接 Review 和自动合并。
- 最小差异：第一次只创建必要文件，不做无关重构。
- 一切可追溯：每一步都要记录“改了什么、跑了什么、结果如何、还剩什么风险”。

推荐执行顺序

阶段	目标	Agent 需要做的事	完成标准
阶段 0 仓库检查	识别栈与现状	检查 package.json / pyproject.toml / go.mod / pom.xml 等；找出现有测试命令；检查 git remote、默认分支、gh 是否可用	输出一页检查摘要；不先问用户栈信息
阶段 1 统一验证	固化唯一验证入口	创建或修正 scripts/verify.sh；必要时改为调用 Makefile / just / task；确保在本地能跑	本地验证至少能稳定复现当前通过或失败状态
阶段 2 仓库内 Agent 规则	让 Agent 有可执行边界	新增或更新 AGENTS.md、.cursor/rules、可选 PR 模板	后续任何 Agent 进仓库都知道该怎么做

阶段 3 本地提需求入口	让 Cursor 能“一句话发车”	新增 .cursor/commands/ship.md；可选新增 scripts/ship_requirement.sh	本地能从需求走到 PR
阶段 4 云端 CI	让 GitHub 替主分支把关	新增 .github/workflows/ci.yml，让它调用相同 verify 命令	PR 页面能看到 CI 检查
阶段 5 Review 与 Merge	补齐第二视角与自动合并	启用 Codex Review；开启 auto-merge；配置 main 的 required checks	PR 通过检查后能自动合并或明确只差一个人工按钮
阶段 6 试跑小需求	验证整条链路	挑一个最小需求完整跑一次	GitHub 上出现已 merge 的真实 PR

4. Agent 需要在仓库里创建或修改的内容

4.1 必做文件

文件	目的	内容要求	如果已存在
AGENTS.md	给 Cursor / Codex / 其他 Agent 的仓库级说明	写清目标、禁止事项、验证命令、完成定义、PR 要求	合并已有规则，不要粗暴覆盖
scripts/verify.sh	统一验证入口	必须串起 install / lint / test / build 中与项目相关的核心步骤	优先复用已有 Makefile / task runner
.cursor/rules/00-workflow.mdc	固定 Cursor 在本仓库的行为边界	要求先计划、最小改动、改动后必须跑验证、禁止直接改 main	如果已有 rules，追加而不是冲掉
.cursor/commands/ship.md	让用户未来一句话触发整个本地流程	分支命名、实现、验证、提交、push、PR、auto-merge	可与现有命令合并
.github/workflows/ci.yml	把本地验证搬到 GitHub	checkout + 环境准备 + 调用 verify 脚本	尽量复用已有 workflow
.github/PULL_REQUEST_TEMPLATE.md	让 PR 说明标准化	改了什么、为什么改、风险、验证结果	没有则新增

4.2 verify.sh 的决策规则

- 如果仓库是 Node / 前端项目：优先查 package.json 中的 scripts，尽量执行 install、lint、test、build 的稳定子集。
- 如果仓库是 Python：优先查 pyproject.toml、requirements.txt、pytest、ruff、mypy 等既有工具。
- 如果仓库是 Go：优先 go test ./...，必要时加 go vet。

- 如果仓库是 Java / Kotlin：优先 Maven 或 Gradle 的测试与构建任务。
- 如果仓库是多子项目混合栈：顶层 verify.sh 负责顺序调用各子项目的 verify，而不是让 Agent 每次现猜。

建议默认模板（Node 版）

```
#!/usr/bin/env bash
set -euo pipefail

npm ci
npm run lint
npm test
npm run build
```

建议默认模板（Python 版）

```
#!/usr/bin/env bash
set -euo pipefail

python -m pip install -U pip
[ -f requirements.txt ] && pip install -r requirements.txt
ruff check .
pytest -q
```

4.3 AGENTS.md 至少应写明的内容

1. 本仓库目标：把需求变成安全可追踪的 PR 和合并结果。
2. 禁止事项：禁止直接 push main，禁止跳过验证，禁止大规模无关重构，禁止修改敏感配置除非任务明确要求。
3. 唯一验证命令：明确写 scripts/verify.sh 或等价命令。
4. 测试要求：逻辑改变时必须补或改测试。
5. PR 要求：标题格式、说明字段、风险说明、验证结果。
6. 完成定义：代码可运行，验证通过，PR 已创建，等待或已完成 merge。

4.4 Cursor 项目命令 ship.md 应具备的动作顺序

1. 先读取 AGENTS.md 与仓库规则，生成简短计划。
2. 从 main 创建 feature 或 fix 分支。
3. 在最小必要范围内实现需求，并更新测试。
4. 运行 verify 脚本；失败则进入调试循环，直到通过。

5. 整理变更摘要与风险点。

6. 提交代码并 push。

7. 通过 gh 创建 PR 指向 main。

8. 如果仓库已开启 auto-merge，则用 squash auto-merge。

4.5 GitHub Actions 的最小要求

- CI workflow 必须以调用 verify 脚本为终点，而不是写出一套与本地不一样的新逻辑。
- 如果项目需要特定语言环境（如 setup-node 或 setup-python），在 workflow 中显式声明。
- 如果测试依赖环境变量或第三方服务，优先用 GitHub Secrets、服务容器或 Mock；不要把真实密钥写入仓库。

参考最小 CI 结构

```
name: ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Verify
        run: bash ./scripts/verify.sh
```

5. 人类一次性配置项（Agent 不能假装自己已经做完）

以下事项可能需要用户在 GitHub 网页、Cursor 登录界面或 Codex 控制台中亲自点选。Agent 应输出清单并给出最短操作路径。

项目	建议状态	备注
GitHub 仓库 Auto-merge	开启	如果 gh 权限足够，可由命令开启；否则让用户在仓库设置中开启
main 分支保护	至少要求 PR + required checks	第一次为了提高成功率，可先不要求人工 Approving Review
Required checks	至少包含 CI 验证	如果后续接入 AI review as check，再把它也设为 required
Codex GitHub Review	连接 GitHub 并启用自动 review	如果当前计划/组织未开放，先降级为 PR 中手动触发 review
Cursor / Codex 登录	本机已登录可用	GUI 登录动作一般需要用户本人完成

建议的 GitHub 规则

- Require a pull request before merging：开启。
- Require status checks to pass before merging：开启，并选择 CI 工作流。
- Require branches to be up to date before merging：建议开启。
- 第一次不强制 Require approving review：这样整条链更容易自动 merge；等流水线稳定后再决定是否引入人工审批。
- Auto-delete head branches：可开启，保持仓库整洁。

如果 Agent 有命令权限，可优先执行的命令

```
gh auth status
gh repo edit --enable-auto-merge
gh pr create --base main --head <branch>
gh pr merge --auto --squash <pr-url>
```

6. 首次试跑要求

第一次不要用大功能验证整条链路。Agent 应主动选择一个“小而真实”的需求，例如：修一个文案、补一个极小缺陷、为现有函数补单测、修一次 lint / build 配置。

1. 从 main 拉出新分支。
2. 完成最小改动并跑 verify。
3. 提交并 push。
4. 创建 PR，等待 GitHub CI 与 Codex Review。
5. 检查 required checks 全绿后，观察 auto-merge 是否生效。
6. 把结果总结为：PR 链接、变更文件、执行命令、验证结果、剩余手工事项。

首次试跑的验收清单

- GitHub 上出现 1 个真实 PR。
- PR 页面可以看到 CI 检查。
- CI 与本地验证结果一致，没有出现“本地过、云端挂”。
- PR 合并到 main，或只差明确的一步人工开关。
- 仓库中的新文件命名清晰、说明充分，下一次提需求时可以复用。

7. 可直接喂给 Agent 的提示词

7.1 给 Cursor 本地 Agent 的提示词

```
你现在是 dimos_huazhijian 仓库的 Orchestrator Agent。

目标：
把当前仓库配置成“提需求 -> 改代码 -> 本地验证 -> push 分支 -> 创建 PR -> 云端检查 -> 自动合并”的稳定流水线。

执行要求：
1. 先检查仓库并识别技术栈，不要先问我技术栈。
2. 优先创建或修正：AGENTS.md、scripts/verify.sh、.cursor/rules/00-workflow.mdc、
   .cursor/commands/ship.md、.github/workflows/ci.yml、可选 PR 模板。
3. 任何改动后都运行 verify 脚本；失败则进入调试循环，直到通过或给出阻塞原因。
4. 不允许直接 push main；必须走功能分支和 PR。
5. 如果有 GitHub / Codex 网页动作无法自动完成，请输出“人类一次性待办清单”，不要假装完成。
6. 最后输出：变更文件列表、执行命令、验证结果、剩余手工步骤。
```

7.2 给 Codex Review Agent 的提示词

```
请对当前 PR 做 Review，重点检查：

- 回归风险
- 漏测
- 破坏性改动
- 安全问题
- 配置错误
- 是否遵守 AGENTS.md 中的规则

请按以下格式输出：
1. 是否建议合并：是 / 否
2. 阻塞项：逐条列出
3. 非阻塞建议：逐条列出
4. 建议补充的测试：逐条列出
```

8. 不可违反的底线

- 不要把密钥、Token、私密配置直接写进仓库。
- 不要为了“让 CI 变绿”而删除真实测试、注释掉关键逻辑或把失败改成跳过。
- 不要在未确认影响范围的情况下修改数据库结构、生产部署流程或公共 API。
- 不要无理由重命名大批文件、迁移目录或做与需求无关的代码美化。
- 如果发现仓库已有较大历史债务，先把当前目标缩小到“建立流程骨架并跑通一条小需求”，不要在第一次就清理所有旧问题。

结语

本说明书的核心不是“让很多 AI 同时聊天”，而是把开发流程拆成清晰、可校验、可追溯的几个阶段。Agent 的第一优先级不是炫技，而是让 dimos_huazhijian 尽快拥有一条可以稳定重复执行的交付链。